



**Después de proteger la
infraestructura y los datos,
deberías preocuparte por tus
modelos de IA.**

**Introducción al Adversarial
Machine Learning.**



Quiénes somos



Miguel Hernández Boza

Security Content Engineer

Sysdig

[@MiguelHzBz](https://twitter.com/MiguelHzBz)



José Ignacio Escribano Pablos

Security & Machine Learning Researcher

BBVA Next Technologies

[/in/josé-ignacio-escribano-pablos](https://in.josé-ignacio-escribano-pablos)

Disclaimer



Todo lo que se muestra en esta charla es con **finés educativos**.

Nuestras opiniones personales no están relacionadas con nuestros actuales centros de trabajo.



“

*“Through 2022, 30% of all AI cyberattacks will leverage **training-data poisoning**, **AI model theft**, or **adversarial samples** to attack AI-powered systems.”*

<https://www.microsoft.com/security/blog/2020/10/22/cyberattacks-against-machine-learning-systems-are-more-common-than-you-think/>

Índice

- 1. Machine Learning y ciberseguridad**
- 2. Machine Learning 101**
- 3. Adversarial Machine Learning**
 - 3.1. Herramientas y frameworks**
 - 3.2. Ataques y demos**
- 4. Conclusiones**



1.

Machine Learning y ciberseguridad



AI Teams

Facebook's 'Red Team' Hacks Its Own AI Programs

<https://www.wired.com/story/facebooks-red-team-hacks-ai-programs>



Blue Team



MLSEC ADVERSARIAL LEARNING VULNERABILITY DISCOVERY MALWARE ANALYSIS DATA ANALYSIS

MACHINE LEARNING FOR COMPUTER SECURITY

Open-source software and datasets developed by the research group of Konrad Rieck

MALWARE ANALYSIS

Drabin — Dataset of Malicious Android Applications

The Drabin dataset consists of roughly 4,000 malicious Android applications that have been collected as part of the Mobile Sandbox project between 2010 and 2014. The dataset can be used to experiment with Android malware and compare different detection approaches. [Data](#) [Paper](#)

Adagio — Structural Analysis and Detection of Android Malware

Adagio is a collection of Python modules for analyzing and detecting Android malware. These modules allow to extract labeled call graphs from Android APKs or DEX files and apply an explicit feature map that captures their structural relationships. Additional modules provide classes for designing binary or multiclass classification experiments and applying machine learning for detection of malicious structure. [Code](#) [Paper](#)

Malheur — Automatic Analysis

Malheur is a tool for the automatic analysis of malware and the detection of malware with similar behavior to a given sample. [Code](#) [Data](#) [Paper](#)

VULNERABILITY DISCOVERY

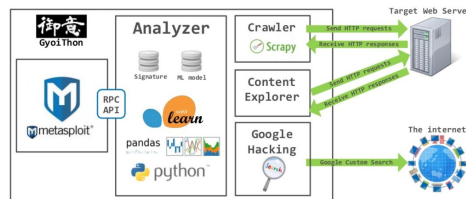
Joern — A Robust Tool for Static Code Analysis

Joern is a platform for robust analysis of C/C++ code. It generates code property graphs, a novel graph representation of code that exposes the code's syntax, control-flow, data-flow and type information. Code property graphs are stored in a graph database. This allows code to be mined using search queries formulated in the graph traversal language Gremlin. Joern forms the basis for assisted vulnerability discovery using machine learning techniques. [Code](#) [Paper](#)

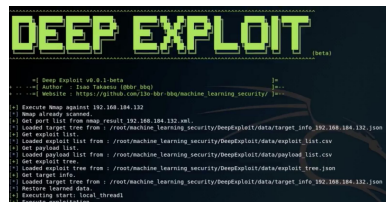
Pulsar — Protocol Learning, Simulation and Stateful Fuzzing

Pulsar is a network fuzzer with automatic protocol learning and simulation capabilities. The tool allows to model a protocol through machine learning techniques, such as clustering and hidden Markov models. These models can be used to simulate communication between Pulsar and a real client or server thanks to semantically correct messages which, in combination with a series of fuzzing primitives, allow to test the implementation of an unknown protocol for errors in deeper states of its protocol state machine. [Code](#) [Paper](#)

Red Team



<https://github.com/gyoisamurai/Gyoithon>

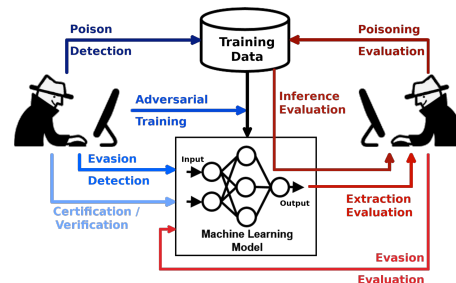


https://github.com/130-bbr-bbq/machine_learning_security



<https://medium.com/analytics-vidhya/deepfake-ai-explainer-and-examples-6d9e4bec55c0>

AI Red Team





The Quiet Victories and False Promises of Machine Learning in Security

<https://sysdig.com/blog/machine-learning-in-cybersecurity/>

The MLSecOps Top 10

The [MLSecOps Top 10](https://ethical.institute/security.html) is an initiative that aims to further the field of machine learning security by identifying the top 10 most common vulnerabilities in the machine learning lifecycle. This project aims to provide an evaluation of security vulnerabilities analogous to the ["OWASP Top 10 Report"](https://ethical.institute/security.html) but with a focus on machine learning security.

<https://ethical.institute/security.html>



SECURING YOUR MACHINE LEARNING MODELS

PHIL BASFORD
MAY 2022

<http://thethirstyrobot.co.uk/OWASP-Suffolk-Securing-ML>



2. **Machine Learning 101**

Machine Learning 101

Inteligencia Artificial (AI)

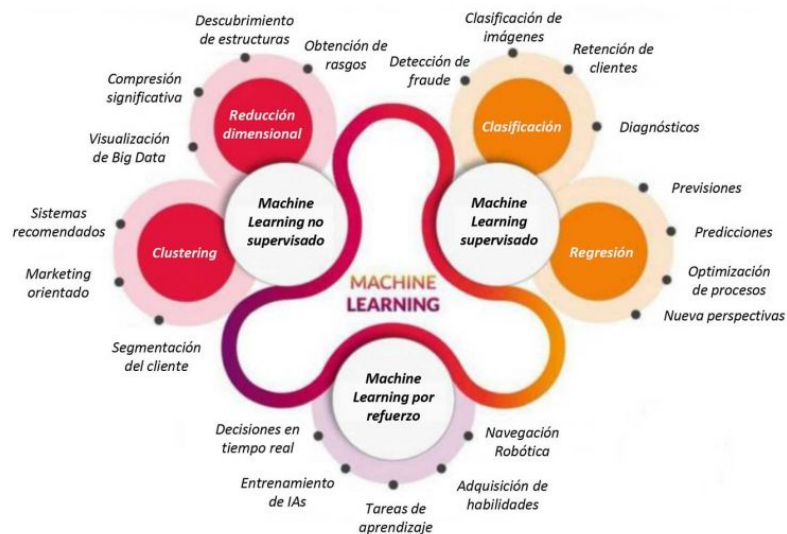
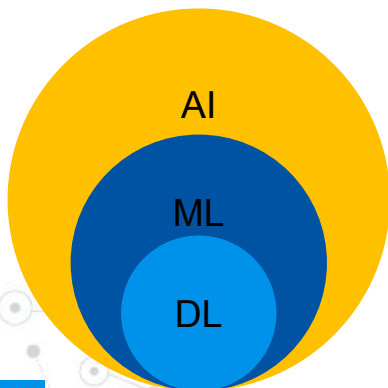
Rama de las Ciencias de la Computación que se ocupa de la **simulación del comportamiento inteligente** en las computadoras.

Machine Learning (ML)

Disciplina de la Inteligencia Artificial relacionada con la implementación de **software que puede aprender de forma autónoma**.

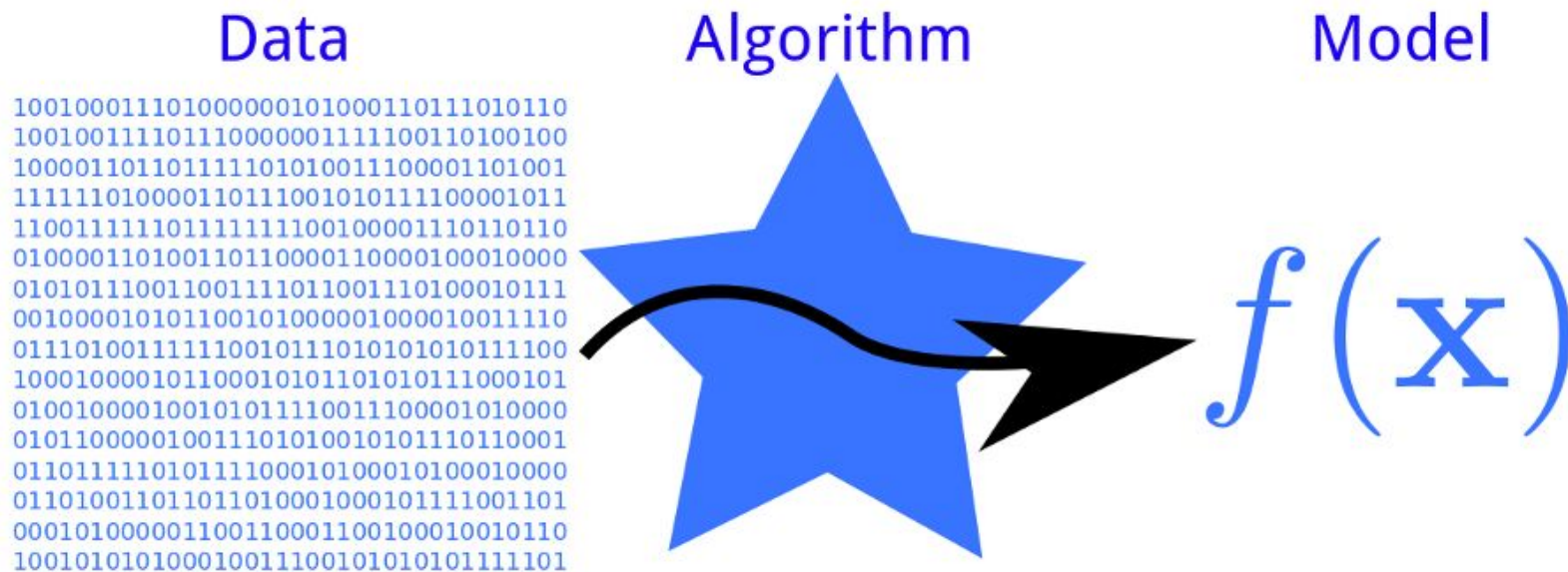
Deep Learning (DL)

Campo del Machine Learning que trabaja con algoritmos **inspirados en la estructura y funciones del cerebro humano**, conocidos como **redes neuronales**.



<https://www.interempresas.net/MetalMecanica/Articulos/347471-Los-conceptos-de-Machine-Learning-y-Deep-Learning-en-la-industria.html>

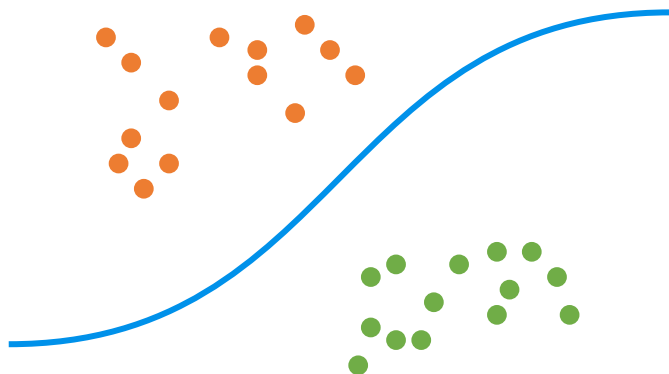
Machine Learning = Datos + Algoritmos



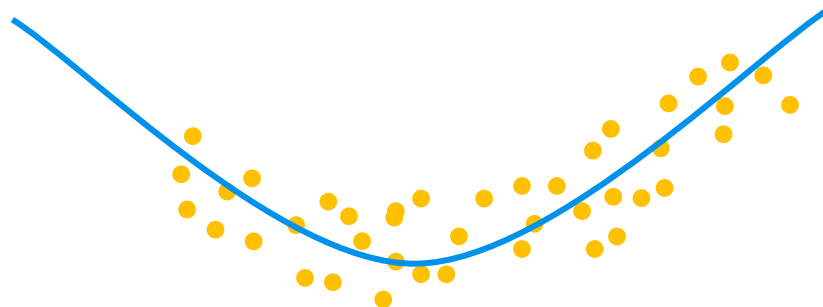
<http://phdp.github.io/posts/2013-07-05-dtl.html>

Clasificación y regresión

Clasificación



Regresión



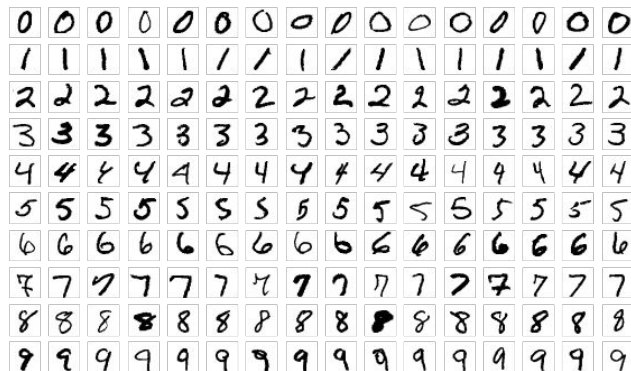
Datos y algoritmos

Repositorios de datos

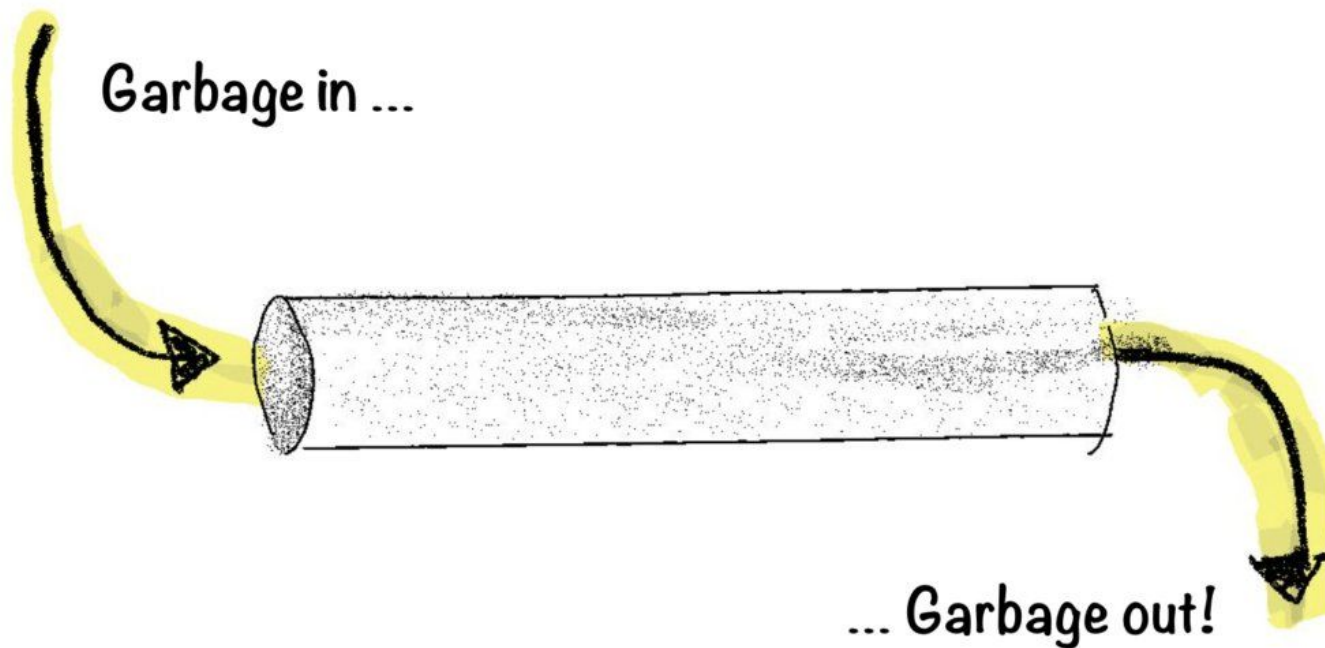
- <https://datasource.kapsarc.org/pages/home> (energía, demografía, economía, ...).
- <https://www.kaggle.com/datasets> (casi cualquier temática).
- <http://usgovxml.com> (servicios web del gobierno de EEUU).
- <https://aws.amazon.com/es/datasets> (varias temáticas).
- <https://www.re3data.org> (búsqueda por palabras).
- <https://www.reddit.com/r/datasets> (publicación y solicitud de distintos conjuntos de datos).
- <http://www.secrepo.com> (conjuntos de datos de seguridad).

Repositorios de imágenes

- **MNIST**: dígitos escritos a mano.
- **CIFAR10/CIFAR100**: imágenes de 32x32 con 10/100 categorías de objetos.
- **CelebA**: 200.000 imágenes con más de 40 atributos de famosos.
- **ImageNet**.
- **COCO** dataset.
- ...

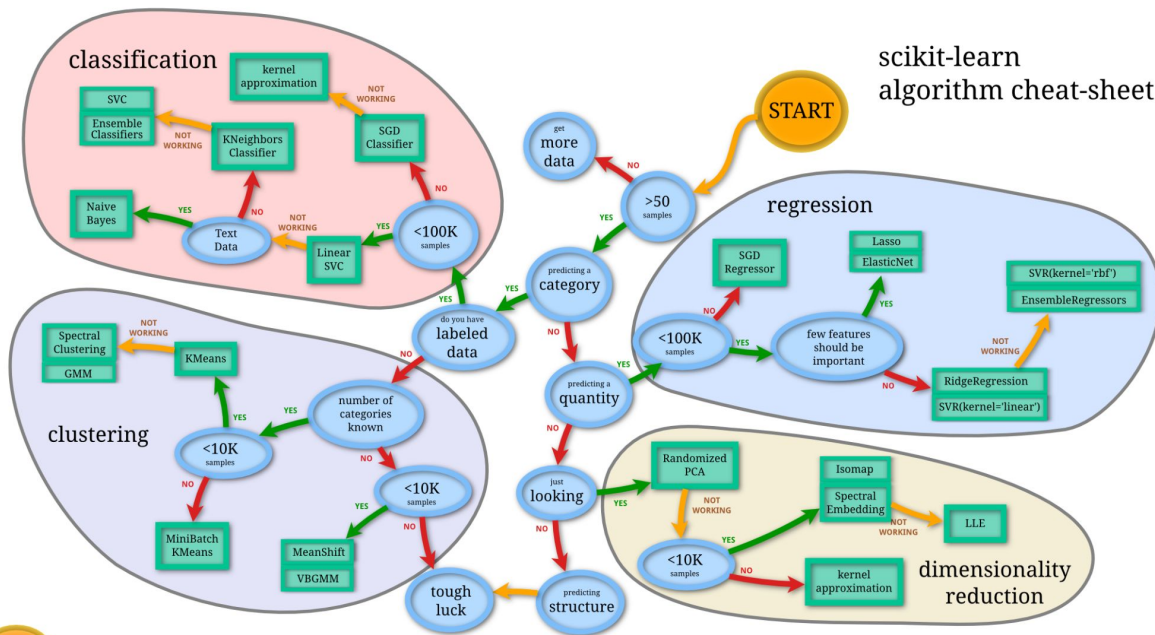


Datos y algoritmos



<https://blog.owulveryck.info/2021/06/08/pov-a-streaming/communication-platform-for-the-data-mesh.html>

Datos y algoritmos

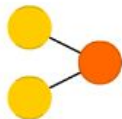


Deep Learning

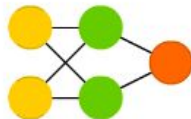
A mostly complete chart of Neural Networks

©2019 Fjodor van Veen & Stefan Leijnen asimovinstitute.org

Perceptron (P)



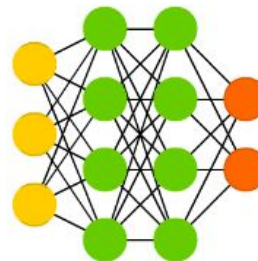
Feed Forward (FF)



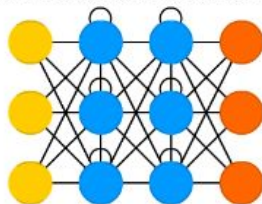
Radial Basis Network (RBF)



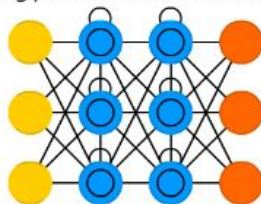
Deep Feed Forward (DFF)



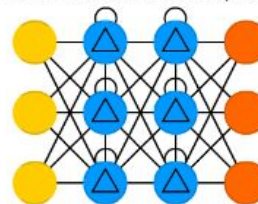
Recurrent Neural Network (RNN)



Long / Short Term Memory (LSTM)

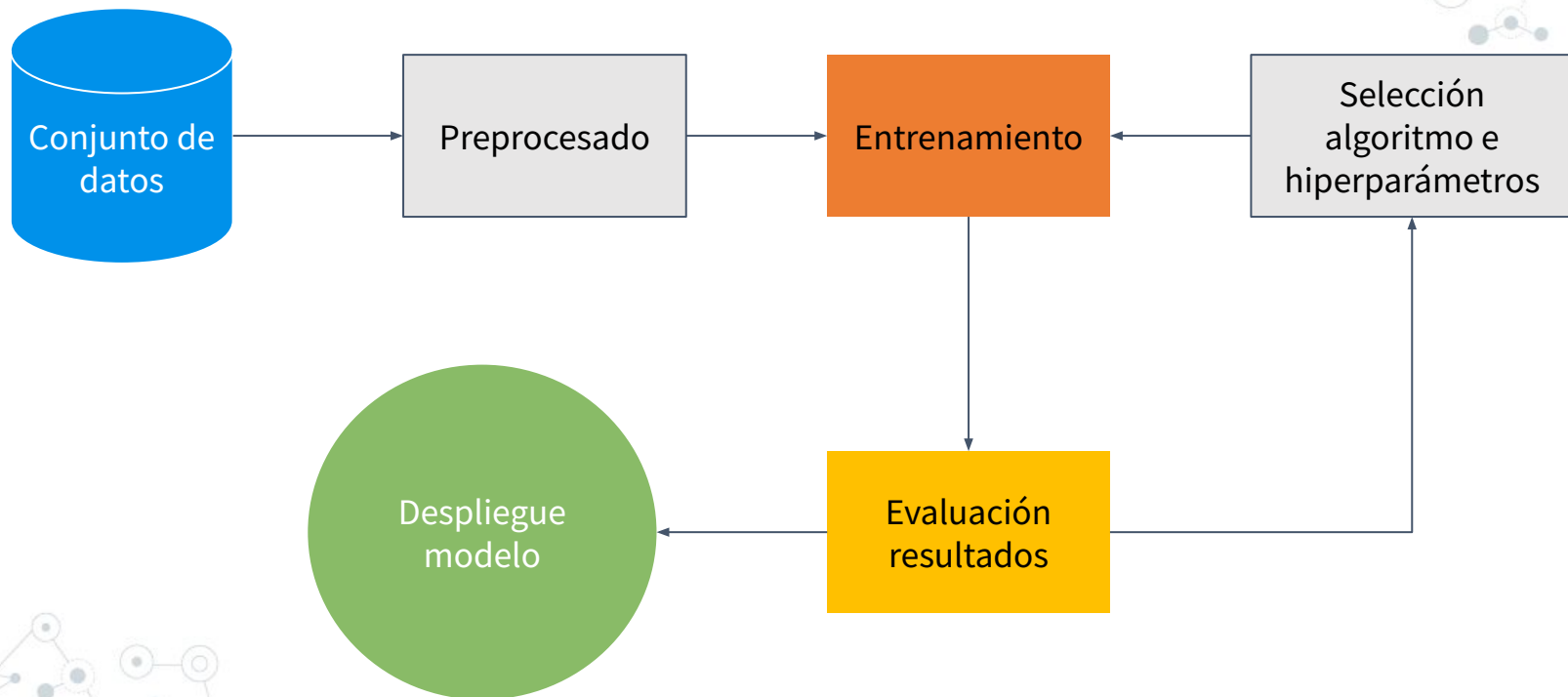


Gated Recurrent Unit (GRU)

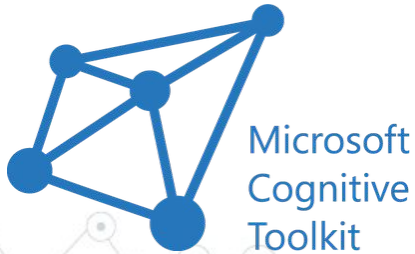


Infografia completa: <https://www.asimovinstitute.org/neural-network-zoo>

Entrenamiento del modelo



Algunas librerías/frameworks

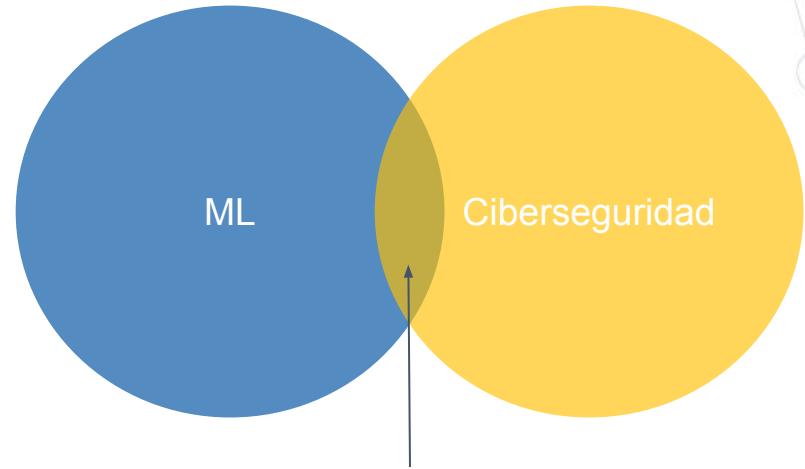




3. **Adversarial Machine Learning**

Adversarial Machine Learning

Rama del machine learning que trata de averiguar los **ataques** que puede sufrir un **modelo** en la presencia de un adversario malicioso y cómo **protegerse de ellos**.



Adversarial Machine Learning



“

*“If you use **machine learning**, there is the risk for exposure, even **though the threat does not currently exist in your space.**”*

*“The **gap** between **machine learning and security** is definitely there.”*

Adversarial Machine Learning

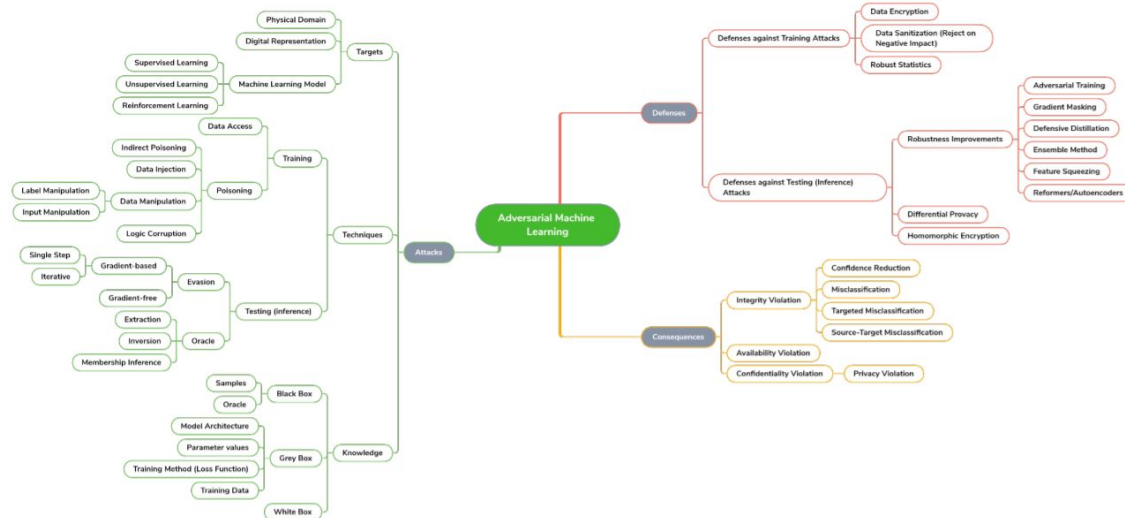
Draft NISTIR 8269

A Taxonomy and Terminology of Adversarial Machine Learning

Elham Tabassi
Kevin J. Burns
Michael Hadjimichael
Andres D. Molina-Markham
Julian T. Sexton

This publication is available free of charge from:
<https://doi.org/10.6028/NIST.IR.8269-draft>

NIST
National Institute of
Standards and Technology
U.S. Department of Commerce



<https://nvlpubs.nist.gov/nistpubs/ir/2019/NIST.IR.8269-draft.pdf>

Adversarial Machine Learning

Reconnaissance	Initial Access	Execution	Persistence	Model Evasion	Exfiltration	Impact
Acquire OSINT information: (Sub Techniques) 1. Arxiv 2. Public blogs 3. Press Releases 4. Conference Proceedings 5. Github Repository 6. Tweets	Pre-trained ML model with backdoor	Execute unsafe ML models (Sub Techniques) 1. ML models from compromised sources 2. Pickle embedding	Execute unsafe ML models (Sub Techniques) 1. ML models from compromised sources 2. Pickle embedding	Evasion Attack (Sub Techniques) 1. Offline Evasion 2. Online Evasion	Exfiltrate Training Data (Sub Techniques) 1. Membership inference attack 2. Model inversion	Defacement
ML Model Discovery (Sub Techniques) 1. Reveal ML model ontology – 2. Reveal ML model family –	Valid account	Execution via API	Account Manipulation		Model Stealing	Denial of Service
Gathering datasets	Phishing	Traditional Software attacks	Implant Container Image	Model Poisoning	Insecure Storage 1. Model File 2. Training data	Stolen Intellectual Property
Exploit physical environment	External remote services			Data Encrypted for Impact Defacement		
Model Replication (Sub Techniques) 1. Exploit API – Shadow Model 2. Alter publicly available, pre-trained weights	Exploit public facing application			Stop System Shutdown/Reboot		
Model Stealing	Trusted Relationship					

<https://github.com/mitre/advmthreatmatrix/blob/master/pages/adversarial-ml-threat-matrix.md#adversarial-ml-threat-matrix>

Adversarial Machine Learning



SOLVING PROBLEMS
FOR A SAFER WORLD®

ABOUT

CENTERS

CAPABILITIES

RESEA

Project Stories

<https://www.mitre.org/publications/project-stories/creating-an-ai-red-team-to-protect-critical-infrastructure>

CREATING AN AI RED TEAM TO PROTECT CRITICAL INFRASTRUCTURE

September 2019

Topics: Homeland Security, Artificial Intelligence, Machine Learning, Cybersecurity, Network Security

As our nation's critical infrastructure increasingly relies upon artificial intelligence, bad actors are finding ways to fool machine learning—with potentially dangerous consequences. Can AI red teams help to protect against such potential attacks?

We need AI Red Teams, NOW!



Tommy Shrove, Ph.D. Follow Edit

Nov 22, 2020 · 6 min read



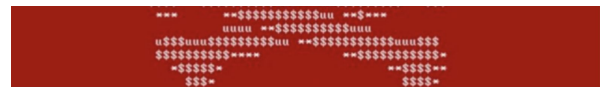
<https://medium.com/disruptive-nerd/we-need-ai-red-teams-now-caaa45af22>

Building an AI Red Team to Stop Problems Before They Start

May 0th 2020

2,872 reads

<https://hackernoon.com/building-an-ai-red-team-to-stop-problems-before-they-start-9e6g32uj>



Featured New

Why the Department of Defense Should Create an AI Red Team

September 7, 2021 · 0 Comments

Estimated Read Time: 5 Minutes

By Rena DeHenre

<https://othjournal.com/2021/09/07/why-the-department-of-defense-should-create-an-ai-red-team/>

Adversarial Machine Learning



This is what the Tesla's AI actually sees in this experiment.



<https://www.nassiben.com/phantoms>

Adversarial Machine Learning



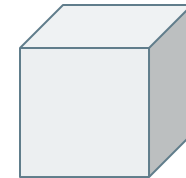
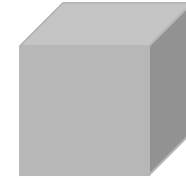
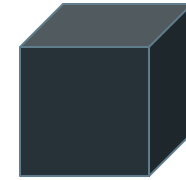
<https://www.csail.mit.edu/news/fooling-neural-networks-w3d-printed-objects>



https://www.youtube.com/watch?v=zQ_uMenoBCk

Adversarial Machine Learning

- **Ataques de caja negra:** el adversario sólo tiene acceso a las entradas y salidas del modelo.
- **Ataques de caja gris:** el ataque se encuentra en un punto intermedio entre los dos tipos de ataques anteriores.
- **Ataques de caja blanca:** el adversario tiene acceso a la arquitectura usada por el modelo, a los datos de entrenamiento, a los parámetros e hiperparámetros.

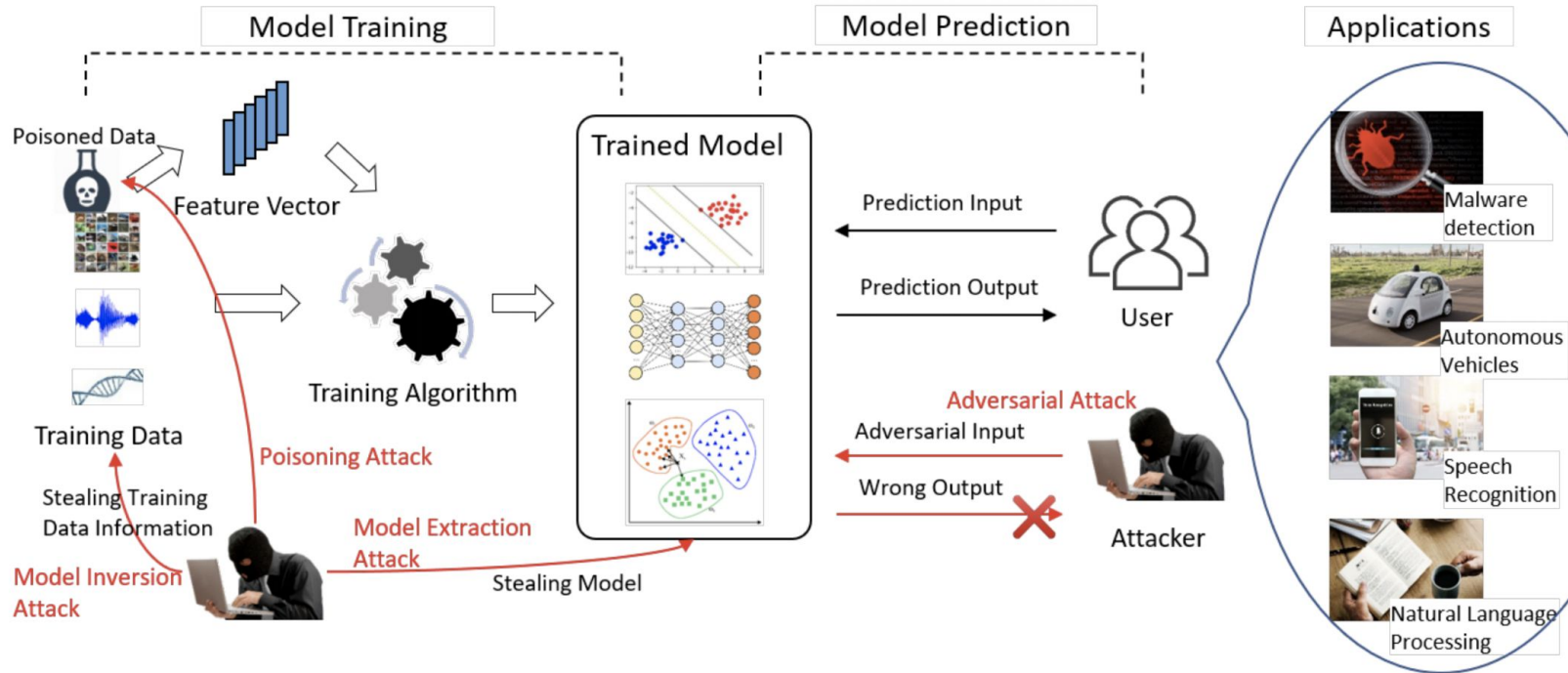


-

+

Conocimiento
del adversario

Adversarial Machine Learning



<https://arxiv.org/abs/1911.12562>

Adversarial Machine Learning

Ataques de **extracción**
(o robo de modelos)

Ataques de **inversión**

Ataques de **envenenamiento**

Ataques de **evasión**

A decorative network diagram in the top-left corner, featuring a complex web of interconnected nodes and lines, with some nodes highlighted in blue and others in grey.

3.1

Herramientas y frameworks

Herramientas y frameworks open source

Nombre	Tipo	Algoritmos	Tipos de ataques	Ataque/Defensa	Frameworks soportados
Cleverhans	Imagen	Deep Learning	Evasión	Ataque	Tensorflow, Keras, JAX
Foolbox	Imagen	Deep Learning	Evasión	Ataque	Tensorflow, PyTorch, JAX
ART	Cualquiera	Deep Learning, SVM, LR, etc.	Todos	Ambos	Tensorflow, Keras, Pytorch, Scikit Learn
TextAttack	Texto	Deep Learning	Evasión	Ataque	Keras, HuggingFace
Advertorch	Imagen	Deep Learning	Evasión	Ambos	----
AdvBox	Imagen	Deep Learning	Evasión	Ambos	PyTorch, Tensorflow, MxNet
DeepRobust	Imagen, grafos	Deep Learning	Evasión	Ambos	PyTorch
Counterfit	Cualquiera	Cualquiera	Evasión	Ataque	----
Adversarial Audio Examples	Audio	DeepSpeech	Evasión	Ataque	----

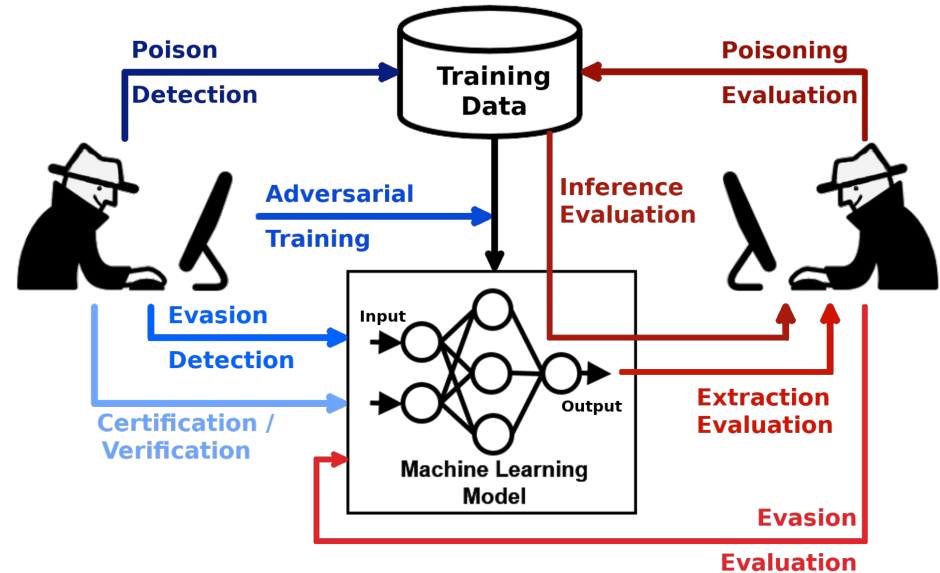
Adversarial Robustness Toolbox (ART)

- Adversarial Robustness Toolbox (<https://github.com/Trusted-AI/adversarial-robustness-toolbox>), abreviado como **ART**, es una librería opensource de Adversarial Machine Learning que permite comprobar la **robustez de los modelos de machine learning**.
- Está desarrollada en **Python** e implementa **ataques y defensas** de extracción, inversión, envenenamiento y evasión.
- ART soporta los **frameworks más populares**: Tensorflow, Keras, PyTorch, MxNet, ScikitLearn, entre muchos otros.
- No está limitada al uso de modelos que emplean **imágenes** como entrada, sino que soporta otros tipos de datos como **audio, vídeo, datos tabulares**, etc.
- Instalación con **pip**.



```
pip install adversarial-robustness-toolbox
```

Adversarial Robustness Toolbox (ART)



Adversarial Robustness Toolbox (ART)

```
!pip install adversarial-robustness-toolbox==1.11.0

import keras
from keras.models import Sequential
from keras.layers import Dense, Flatten, Conv2D, MaxPooling2D
import numpy as np
import matplotlib.pyplot as plt
from art.utils import load_mnist

import tensorflow as tf
tf.compat.v1.disable_eager_execution()

import warnings
warnings.filterwarnings('ignore')

%matplotlib inline

(x_train, y_train), (x_test, y_test), min_pixel_value, max_pixel_value = load_mnist()

print("x_train shape:", x_train.shape)
```

https://github.com/jieep/adversarial-machine-learning/blob/main/notebooks/art_install.ipynb

Counterfit

- Counterfit (<https://github.com/Azure/counterfit>) es una **CLI** escrita en Python y desarrollada por Microsoft.
- Desarrollada para **auditorías de seguridad sobre modelos de ML.**
- Implementa algoritmos de **evasión de caja negra.**
- Se basa en **ART y TextAttack.**

Microsoft

[illegible]

#ATML

```
list targets: lista modelos disponibles
list frameworks: lista frameworks de ataque disponibles
load <framework>: carga el framework
list attacks: lista ataques del framework seleccionado
interact <target>: selecciona el modelo objetivo
predict -i <ind>: predicción sobre la entrada ind
use <attack>: selecciona ataque
run: ejecuta ataque
scan: ejecuta ataque
show options: muestra opciones del ataque
set <param>=<value>: cambia el valor de param a value
save: guarda los resultados en json
```

3.2

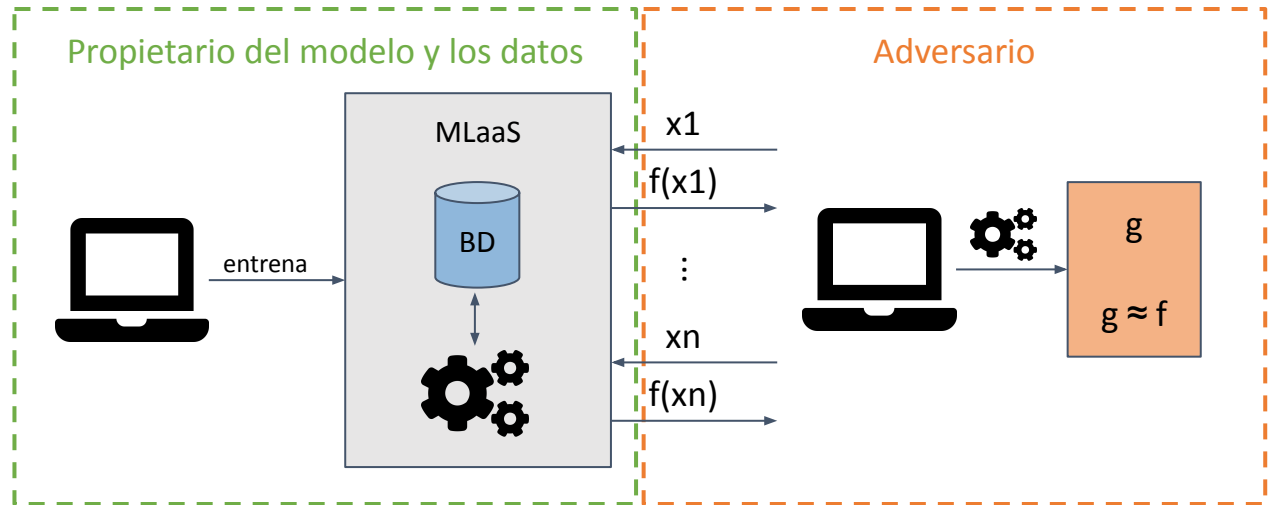
Ataques y demos



<https://github.com/jiep/adversarial-machine-learning>

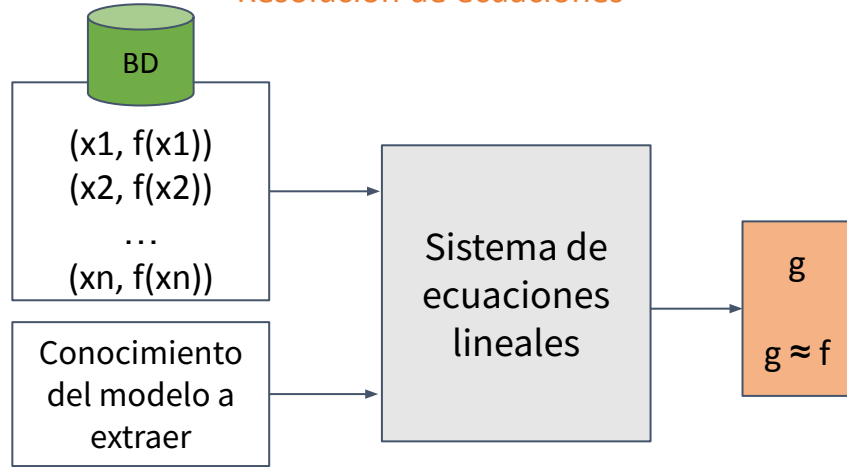
Ataques de extracción (o robo de modelos)

- Permiten a un adversario **robar los parámetros** de un modelo de machine learning.
- Se produce al realizar peticiones al modelo objetivo con **entradas preparadas para extraer la mayor cantidad de información posible** y con el conjunto de entradas y salidas entrenar un modelo llamado modelo sustituto.
- Permite además **realizar otros tipos de ataque** como evasión y/o inversión.

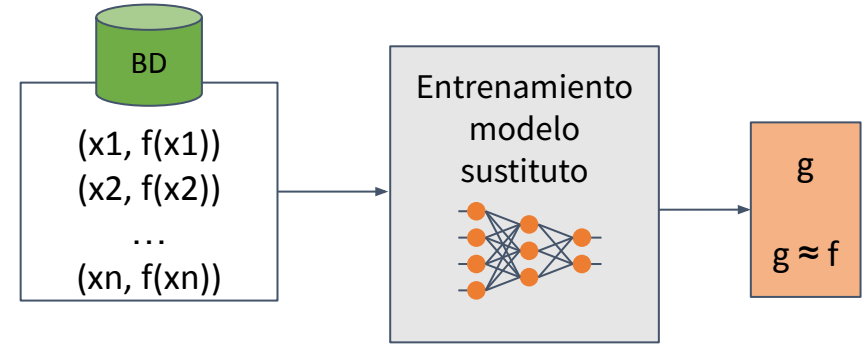


Ataques de extracción (o robo de modelos)

Resolución de ecuaciones



Modelo sustituto



Complejidad del modelo
Conocimiento del modelo

Ataques de extracción (o robo de modelos)

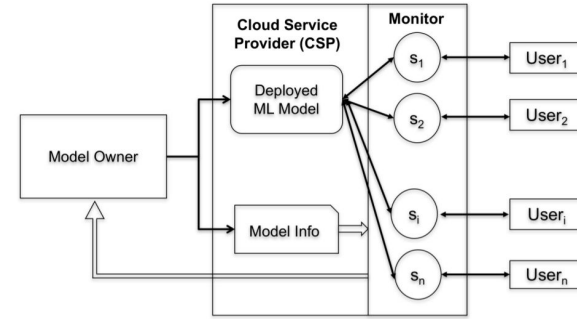
Limitaciones

- Entrenar un **modelo sustituto** es equivalente (en muchos casos) a entrenar un modelo desde cero.
- Muy **costoso** computacionalmente.
- **Limitaciones** en el número de peticiones.

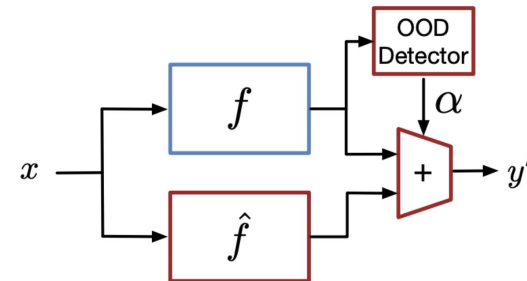
Ataques de extracción (o robo de modelos)

Medidas defensivas

- **Redondeo** de los valores de salida.
- **Privacidad diferencial.**
- **Ensembles.**
- **Defensas específicas**
 - Arquitecturas específicas
(<https://arxiv.org/abs/1711.07221>)
 - PRADA
(<https://arxiv.org/abs/1805.02628>)
 - Adaptive Misinformation
(<https://arxiv.org/abs/1911.07100>)
 - ...

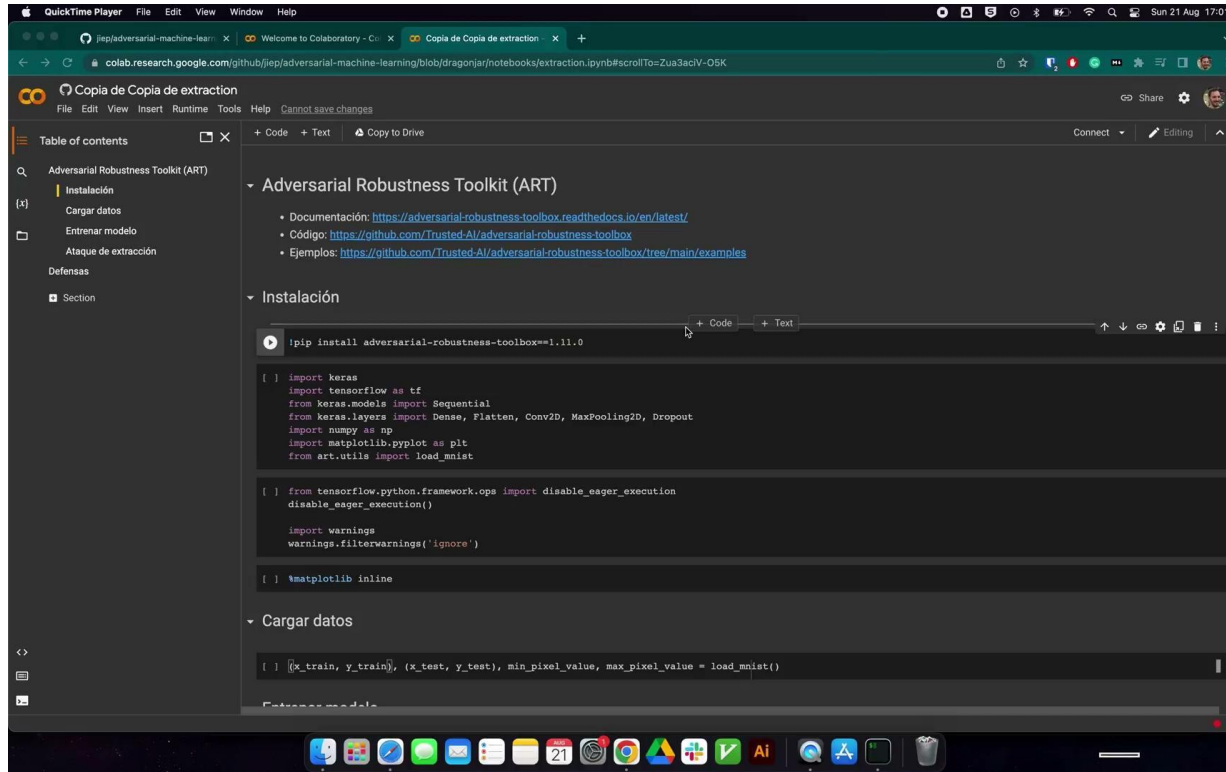


<https://arxiv.org/abs/1711.07221>



<https://arxiv.org/abs/1911.07100>

Ataques de extracción (o robo de modelos)



The screenshot shows a Google Colab notebook interface. The browser address bar displays the URL: <https://colab.research.google.com/github/jieep/adversarial-machine-learning/blob/dragonjar/notebooks/extraction.ipynb#scrollTo=Zua3acIV-05K>. The notebook title is "Copia de Copia de extracción". The left sidebar shows a "Table of contents" with sections: "Adversarial Robustness Toolkit (ART)", "Instalación", "Cargar datos", "Entrenar modelo", "Ataque de extracción", and "Defensas". The main content area is titled "Adversarial Robustness Toolkit (ART)" and includes a list of links for documentation, code, and examples. Below this, the "Instalación" section contains a code cell with the command `!pip install adversarial-robustness-toolbox==1.11.0`. The "Cargar datos" section contains a code cell with imports for keras, tensorflow, numpy, matplotlib, and art, and a line to load the mnist dataset: `(x_train, y_train), (x_test, y_test), min_pixel_value, max_pixel_value = load_mnist()`.

```
!pip install adversarial-robustness-toolbox==1.11.0
```

```
[ ] import keras
import tensorflow as tf
from keras.models import Sequential
from keras.layers import Dense, Flatten, Conv2D, MaxPooling2D, Dropout
import numpy as np
import matplotlib.pyplot as plt
from art.utils import load_mnist

[ ] from tensorflow.python.framework.ops import disable_eager_execution
disable_eager_execution()

import warnings
warnings.filterwarnings('ignore')

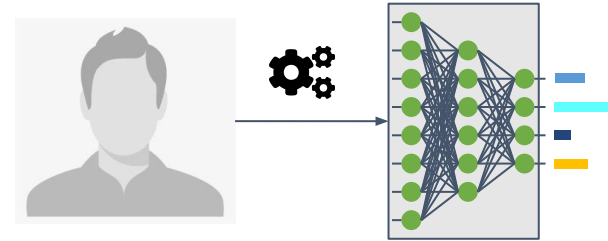
[ ] %matplotlib inline
```

```
[ ] (x_train, y_train), (x_test, y_test), min_pixel_value, max_pixel_value = load_mnist()
```

<https://github.com/jieep/adversarial-machine-learning/blob/main/notebooks/extraction.ipynb>

Ataques de inversión

- Tienen como objetivo **invertir el flujo de información** de un modelo de machine learning.
- Permiten a un adversario tener un **conocimiento del modelo que no pretendía ser compartido de forma explícita**.
- Incluye conocer los **datos de entrenamiento o información como propiedades estadísticas del modelo**.
- Se distinguen tres tipos:
 - **Membership Inference Attack** (MIA): un adversario trata de determinar si una muestra fue empleada como parte del entrenamiento.
 - **Property Inference Attack** (PIA): un adversario trata de extraer propiedades de carácter estadístico que no fueron explícitamente codificadas como características durante la fase de entrenamiento.
 - **Reconstrucción**: un adversario trata de recrear una o más muestras del conjunto de entrenamiento y/o sus correspondientes etiquetas. También llamados de inversión.



←
Inversión del modelo



<https://dl.acm.org/doi/10.1145/2810103.2813677>

Ataques de inversión

Causas del filtrado de información

- El **sobreentrenamiento** es una condición suficiente, pero no necesaria para realizar un ataque **MIA**. Incluso en modelos que generalizan bien es posible realizar un ataque MIA sobre un **subconjunto de los datos de entrenamiento** que son denominados registros vulnerables.
- La **complejidad de los datos y un número alto de clases** incrementa la posibilidad de ataques **MIA**.
- Las propuestas de **entrenamiento adversario robusto** incrementan las posibilidades de ataques **MIA**.
- Un modelo con una **alta potencia predictiva** es más propenso a **ataques de reconstrucción**.

Ataques de inversión

Medidas defensivas

- Uso de la **criptografía avanzada**. Entre las medidas se incluyen la **privacidad diferencial**, la **criptografía homomórfica** y la **computación multiparte segura**.
- Uso de técnicas de regularización como **Dropout** debido a la **relación entre sobreentrenamiento y privacidad**.
- La **compresión de modelos** se ha propuesto como defensa frente a ataques de reconstrucción.

Ataques de inversión

The screenshot shows a Google Colab interface with the following content:

- Table of contents:** Adversarial Robustness Toolkit (ART), Instalación, Cargar datos, Entrenar modelo, Ataque de Inversión.
- Adversarial Robustness Toolkit (ART):**
 - Documentación: <https://adversarial-robustness-toolbox.readthedocs.io/en/latest/>
 - Código: <https://github.com/Trusted-AI/adversarial-robustness-toolbox>
 - Ejemplos: <https://github.com/Trusted-AI/adversarial-robustness-toolbox/tree/main/examples>
- Instalación:**

```
!pip install adversarial-robustness-toolbox==1.11.0
```
- Cargar datos:**

```
[ ] import keras
    from keras.models import Sequential
    from keras.layers import Dense, Flatten, Softmax
    import numpy as np
    import matplotlib.pyplot as plt
    from art.utils import load_mnist

[ ] import tensorflow as tf
    tf.compat.v1.disable_eager_execution

    import warnings
    warnings.filterwarnings('ignore')

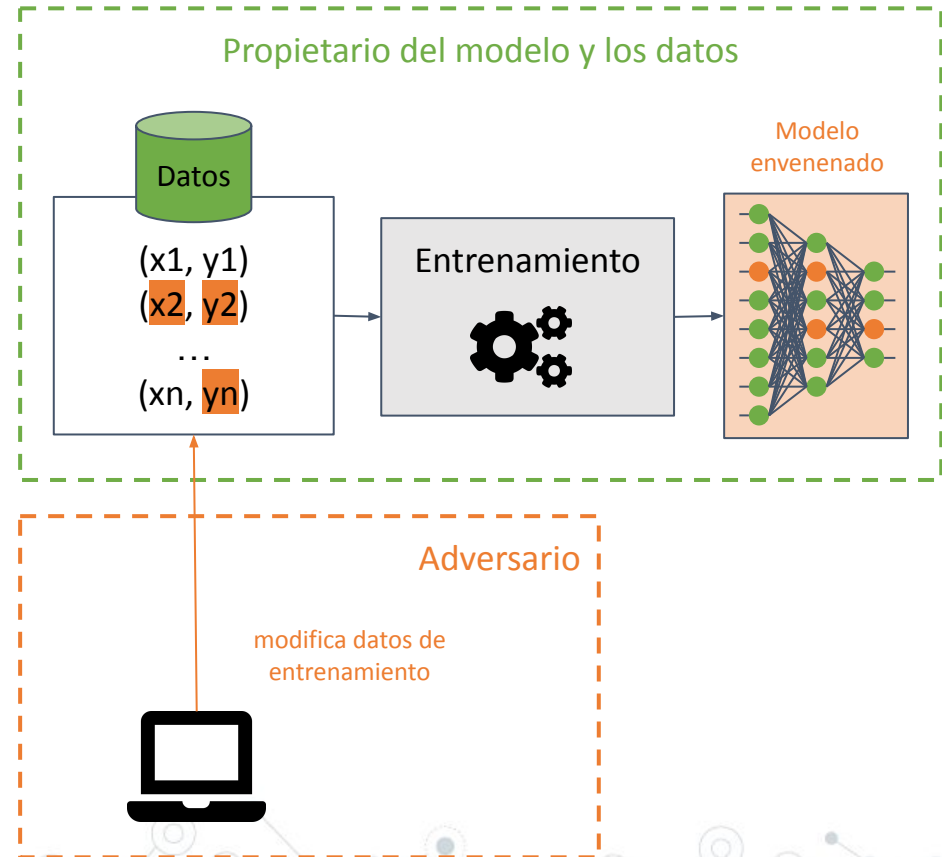
[ ] %matplotlib inline
```
- Entrenar modelo:**

```
[ ] from art.estimators.classification import KerasClassifier
```

<https://github.com/jiep/adversarial-machine-learning/blob/main/notebooks/inversion.ipynb>

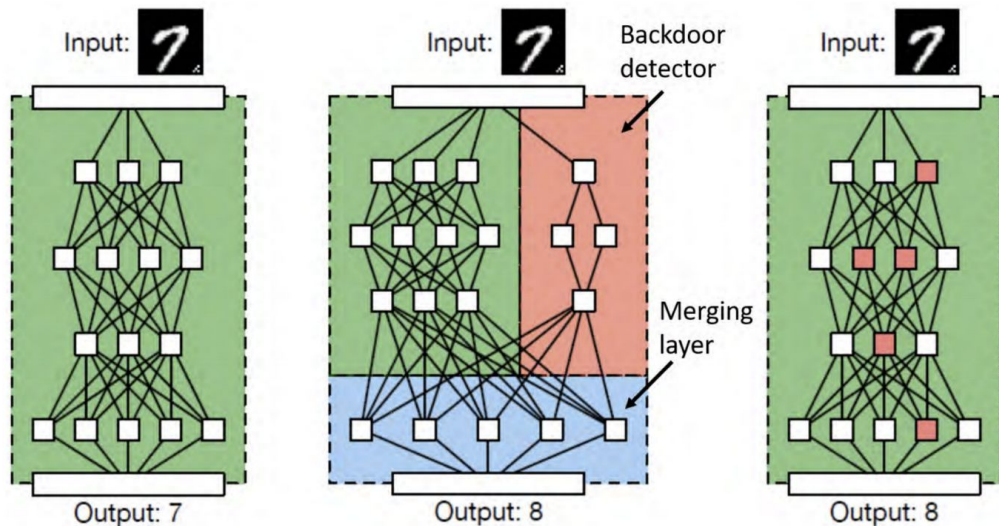
Ataques de envenenamiento

- Buscan **corromper el conjunto de entrenamiento** haciendo que un modelo de machine learning **reduzca su precisión**.
- Este ataque es **difícil de detectar** al realizarse sobre los datos de entrenamiento, puesto que el ataque se puede **propagar entre distintos modelos** al emplear los mismos datos.
- El adversario busca modificar la **frontera de decisión** y, como resultado, produciendo **predicciones incorrectas** o,
- Crear una **puerta trasera** en un modelo. El modelo se comporta de forma correcta (devolviendo las predicciones deseadas) en la mayoría de casos, **salvo en ciertas entradas especialmente creadas por el adversario que producen resultados no deseados**.



Ataques de envenenamiento

BadNets



Disparadores



<https://arxiv.org/abs/1708.06733>

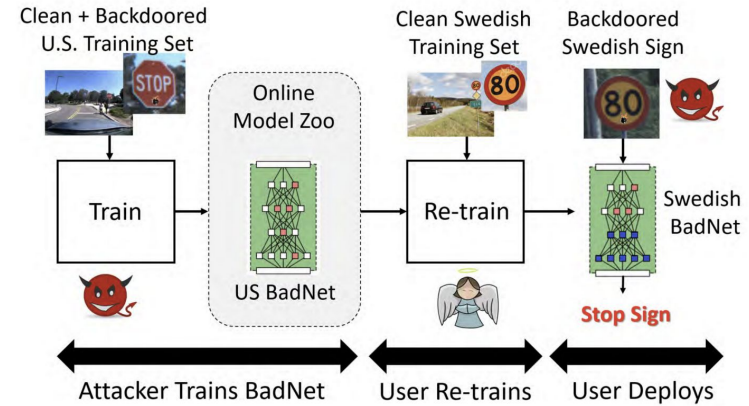
Ataques de envenenamiento



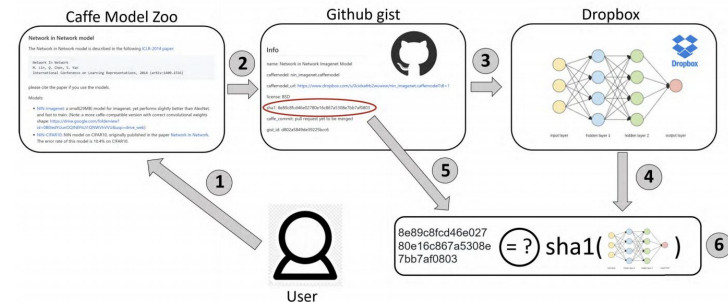
<https://arxiv.org/abs/1708.06733>

Ataques de envenenamiento

- Las BadNets son capaces de **conservarse en un modelo**, aunque se **vuelvan a entrenar de nuevo para otra tarea** distinta a la del modelo original (*transfer learning*).



- Los **modelos preentrenados públicos** pueden contener **puertas traseras**.



<https://arxiv.org/abs/1708.06733>

Ataques de envenenamiento

 IBM Research AI

Game RulesIntro to BackdoorsChallengesAbout the ResearchHome

Fool the AI!

Hackers can use backdoors to poison training data and cause an AI model to misclassify images. Learn how IBM researchers can tell when data has been poisoned, then guess what backdoors have been hidden in these datasets.

Can you guess the backdoor?

PLAY



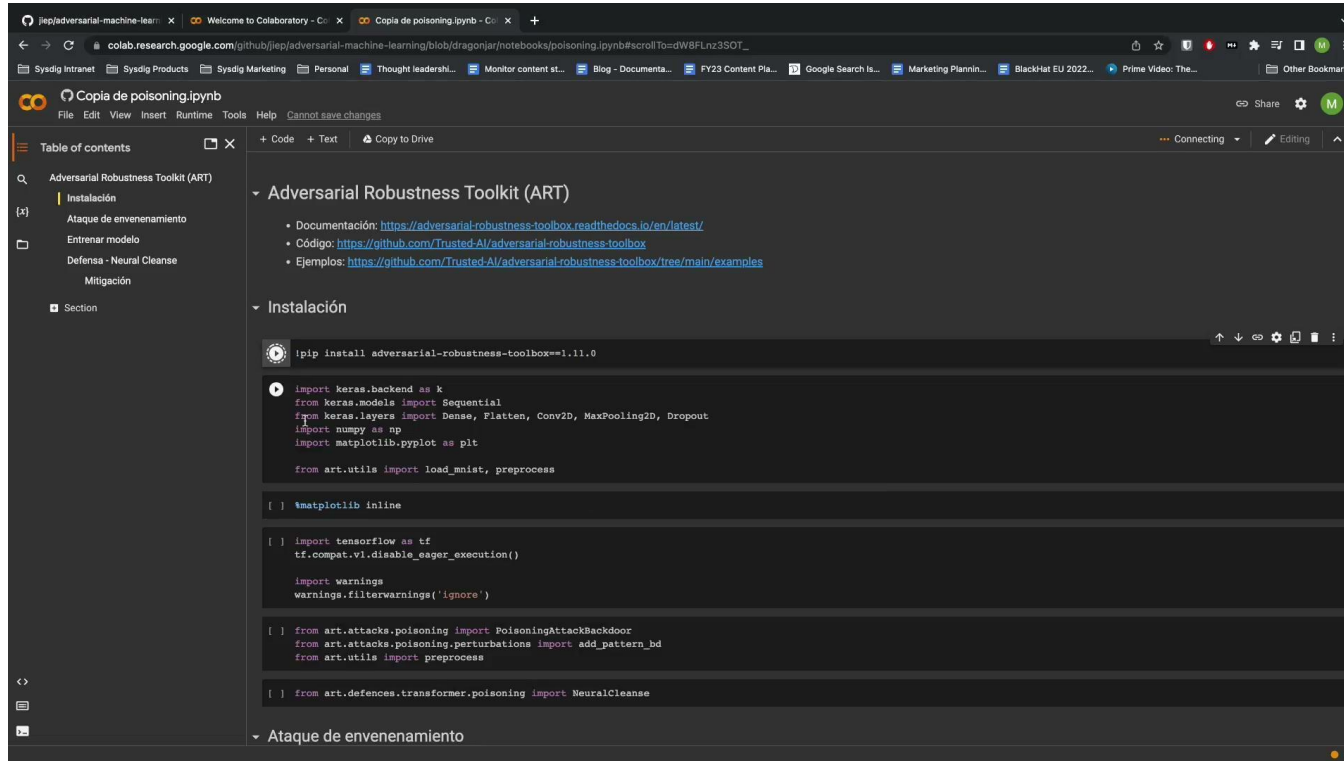
<https://fooltheai.mybluemix.net>

Ataques de envenenamiento

Medidas defensivas

- **Protección del dato**, que incluye evitar la **modificación, la denegación y la falsificación de los datos** y, la **detección de los datos envenenados**, junto con el uso del **saneamiento de datos**.
- **Protección de los algoritmos**, que intenta emplear **métodos robustos de entrenamiento**.
- **Defensas específicas**.

Ataques de envenenamiento



The screenshot shows a Google Colab notebook interface. The browser address bar displays the URL: colab.research.google.com/github/jiep/adversarial-machine-learning/blob/dragonjar/notebooks/poisoning.ipynb#scrollTo=dW8FLnz3SOT_. The notebook title is "Copia de poisoning.ipynb".

The left sidebar shows a "Table of contents" with the following sections:

- Adversarial Robustness Toolkit (ART)
 - Instalación
 - Ataque de envenenamiento
 - Entrenar modelo
 - Defensa - Neural Cleanse
 - Mitigación
- Section

The main content area shows the following sections and code:

Adversarial Robustness Toolkit (ART)

- Documentación: <https://adversarial-robustness-toolbox.readthedocs.io/en/latest/>
- Código: <https://github.com/Trusted-AI/adversarial-robustness-toolbox>
- Ejemplos: <https://github.com/Trusted-AI/adversarial-robustness-toolbox/tree/main/examples>

Instalación

```
!pip install adversarial-robustness-toolbox==1.11.0
```

```
import keras.backend as k
from keras.models import Sequential
from keras.layers import Dense, Flatten, Conv2D, MaxPooling2D, Dropout
import numpy as np
import matplotlib.pyplot as plt

from art.utils import load_mnist, preprocess

[ ] %matplotlib inline

[ ] import tensorflow as tf
    tf.compat.v1.disable_eager_execution()

    import warnings
    warnings.filterwarnings('ignore')

[ ] from art.attacks.poisoning import PoisoningAttackBackdoor
    from art.attacks.poisoning.perturbations import add_pattern_bd
    from art.utils import preprocess

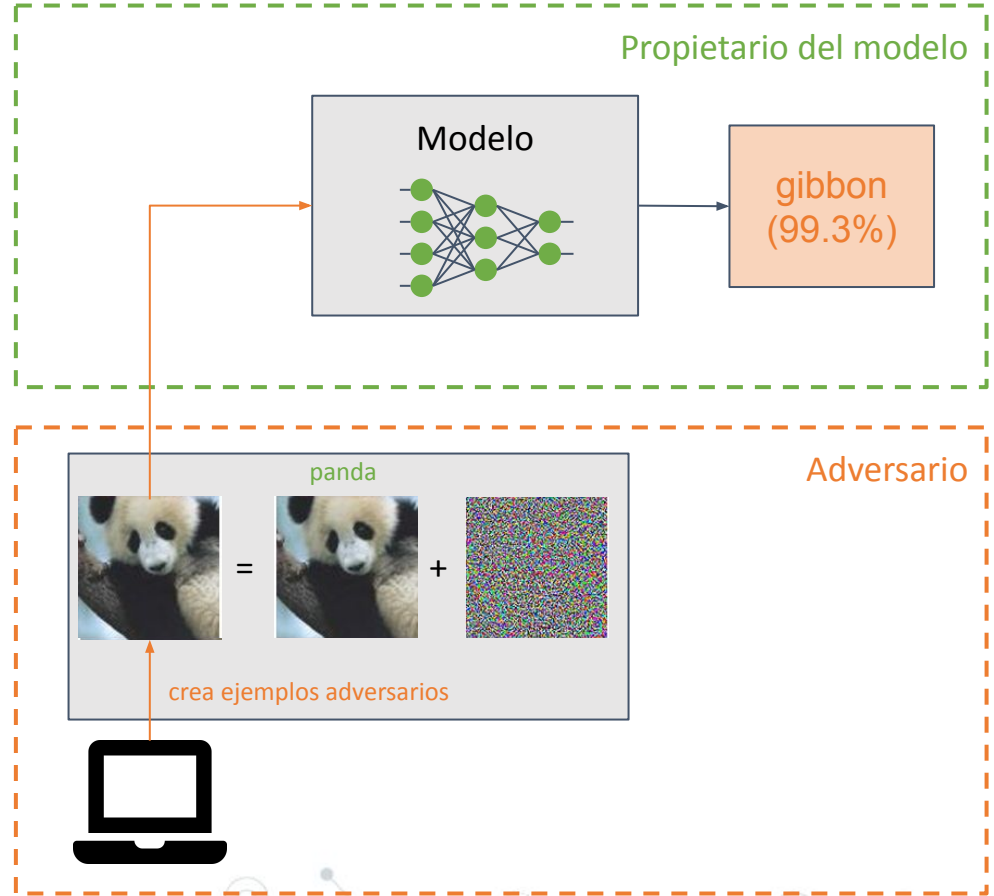
[ ] from art.defences.transformer.poisoning import NeuralCleanse
```

Ataque de envenenamiento

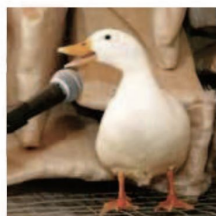
<https://github.com/jiep/adversarial-machine-learning/blob/main/notebooks/poisoning.ipynb>

Ataques de evasión

- Un adversario inserta una **pequeña perturbación** (en forma de ruido) en la entrada de un modelo de machine learning para que **clasifique de forma incorrecta** (ejemplo adversario).
- Son similares a los ataques de envenenamiento, pero su principal diferencia radica en que los ataques de evasión tratan de **explotar debilidades del modelo en la fase de inferencia**.
- El objetivo del adversario es que los ejemplos adversarios sean **imperceptibles por un humano**.



Ataques de evasión



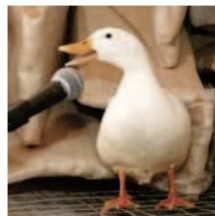
'Duck'

+



$\times 0.07$

=



'Horse'



'How are you?'

+



$\times 0.01$

=



'Open the door'

Classifier: Word-LSTM

Original Text Prediction: Sci/Tech (Confidence = 47.80%)

Adversarial Text Prediction: Business (Confidence = 52.52%)

Original Text: *Tyrannosaurus rex achieved its massive size due to an enormous growth spurt **during** its adolescent years.*

Adversarial Text: *Tyrannosaurus rex achieved its massive size due to an enormous growth spurt **durnig** its adolescent years.*

Classifier: Text-CNN

Original Text Prediction: Company (Confidence = 98.16%)

Adversarial Text Prediction: Artist (Confidence = 20.27%)

Original Text: *Yates is a gardening company **in** New Zealand and Australia.*

Adversarial Text: *Yates is a gardening company **i** New Zealand and Australia.*

<https://arxiv.org/abs/1803.09156>

<https://arxiv.org/abs/2002.00760>

Ataques de evasión

Try it out

1. Select an image to target



2. Simulate Attack

Adversarial noise type
Projected Gradient Descent

Determine strength

None low med high

3. Defend attack

Gaussian Noise

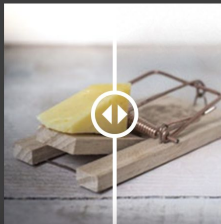
None low med high

Spatial Smoothing

None low med high

Visual Code

Original Modified



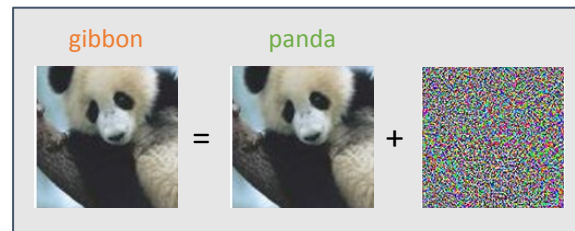
<https://art-demo.mybluemix.net>

Ataques de evasión

- **Dirigidos:** el adversario tiene por objetivo obtener una **predicción de su elección**.



- **No dirigidos:** el adversario tiene por objetivo obtener una **clasificación de forma incorrecta**.

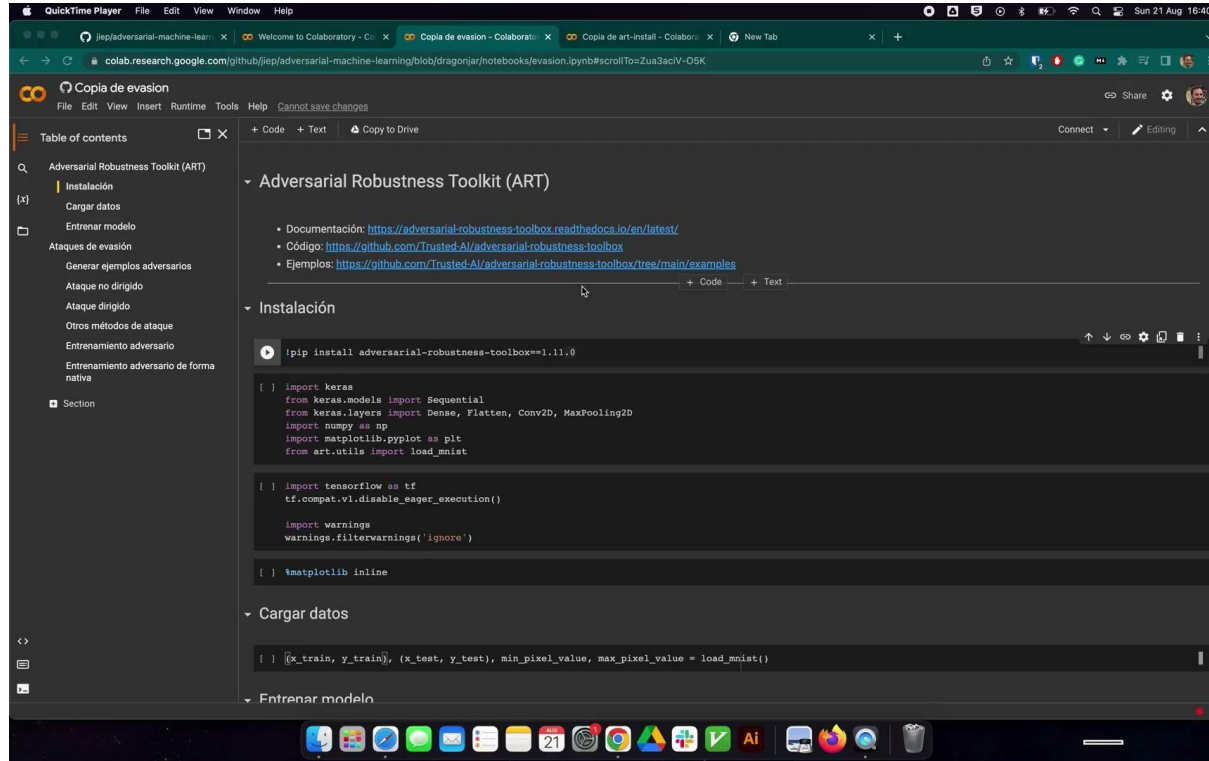


Ataques de evasión

Algunos ataques de caja blanca

- L-BFGS
- FGSM
- BIM
- JSMA
- Carlini & Wagner (C&W)
- NewtonFool
- EAD
- BIM
- UAP
- ...

Ataques de evasión



The screenshot shows a Google Colab notebook interface. The browser address bar displays the URL: colab.research.google.com/github/jiep/adversarial-machine-learning/blob/main/notebooks/evasion.ipynb?scrollTo=Zus3acIV-05K. The notebook title is 'Copia de evasión'. The left sidebar shows a table of contents with sections like 'Adversarial Robustness Toolkit (ART)', 'Instalación', 'Cargar datos', and 'Entrenar modelo'. The main content area shows the following code blocks:

```
Adversarial Robustness Toolkit (ART)

• Documentación: https://adversarial-robustness-toolbox.readthedocs.io/en/latest/
• Código: https://github.com/TrustedAI/adversarial-robustness-toolbox
• Ejemplos: https://github.com/TrustedAI/adversarial-robustness-toolbox/tree/main/examples

Instalación

!pip install adversarial-robustness-toolbox==1.11.0

[ ] import keras
from keras.models import Sequential
from keras.layers import Dense, Flatten, Conv2D, MaxPooling2D
import numpy as np
import matplotlib.pyplot as plt
from art.utils import load_mnist

[ ] import tensorflow as tf
tf.compat.v1.disable_eager_execution()

import warnings
warnings.filterwarnings('ignore')

[ ] %matplotlib inline

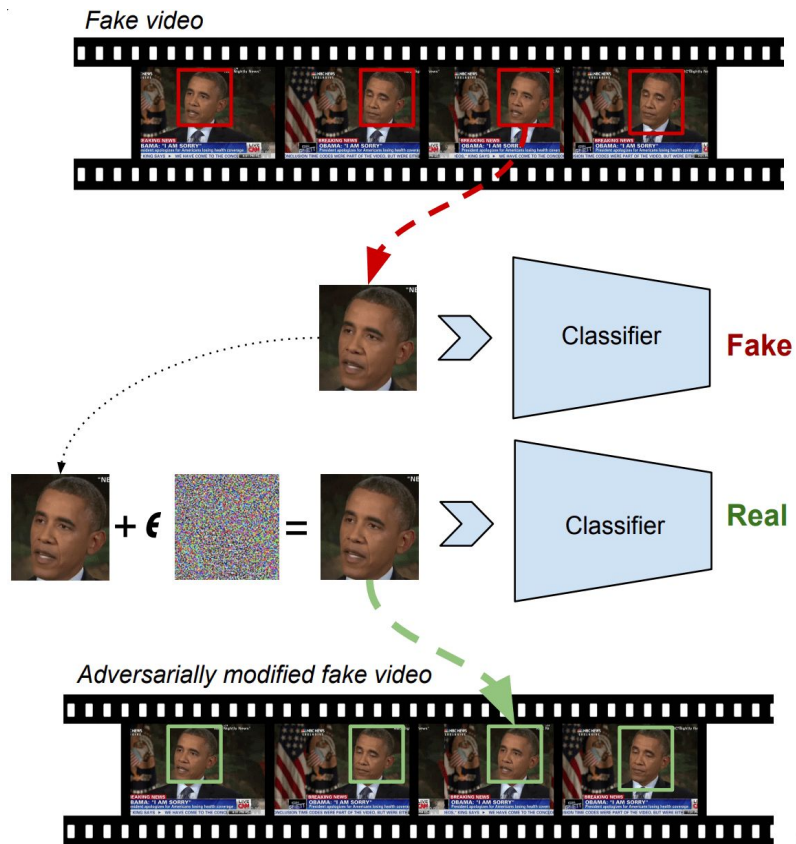
Cargar datos

[ ] (x_train, y_train), (x_test, y_test), min_pixel_value, max_pixel_value = load_mnist()
```

<https://github.com/jiep/adversarial-machine-learning/blob/main/notebooks/evasion.ipynb>

Ataques de evasión

- **Adversarial Deepfakes** (<https://adversarialdeepfakes.github.io>): permite la **evasión de defensas** frente a detección de **deepfakes**. En concreto, **FaceForensics++**.

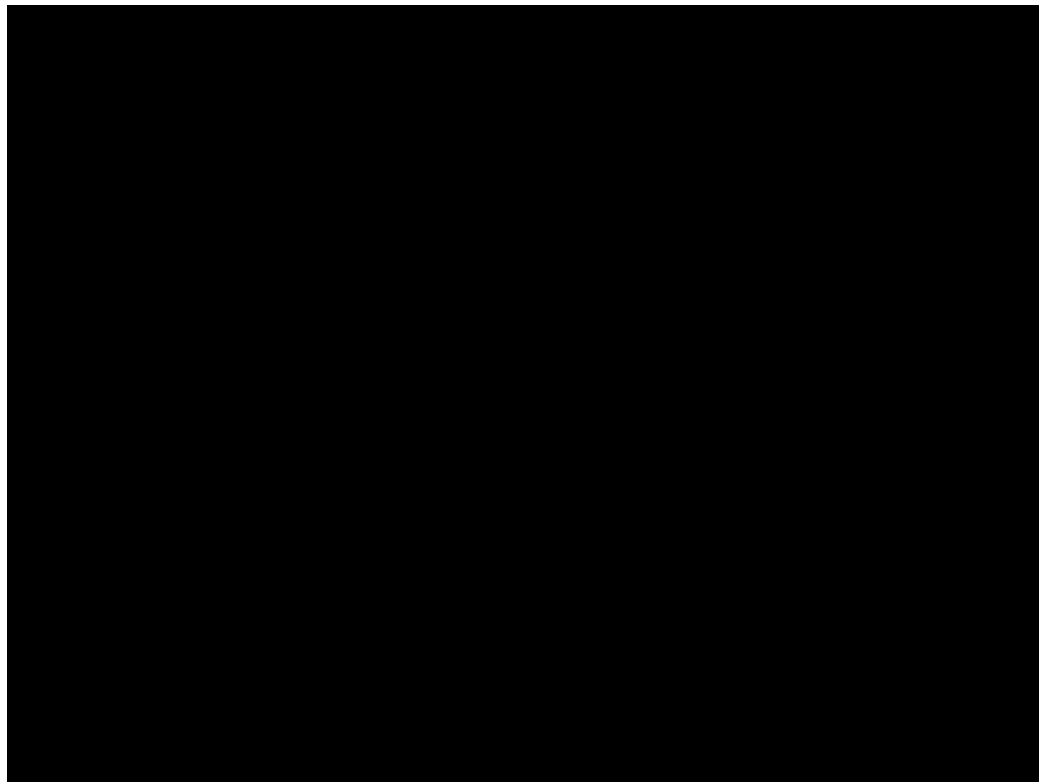


Ataques de evasión

Medidas defensivas

- **Entrenamiento adversario**, que consiste en emplear **ejemplos adversarios durante el entrenamiento** para que el modelo aprenda características de los ejemplos adversarios, **haciendo más robusto el modelo** ante este tipo de ataque.
- **Transformaciones** sobre las entradas.
- **Enmascarado/regularización del gradiente**. No muy efectiva.
- **Defensas débiles**.

Ataques de evasión



Resultado del ataque



<https://github.com/jiep/adversarial-machine-learning/blob/main/notebooks/counterfit.md>



4. Conclusiones



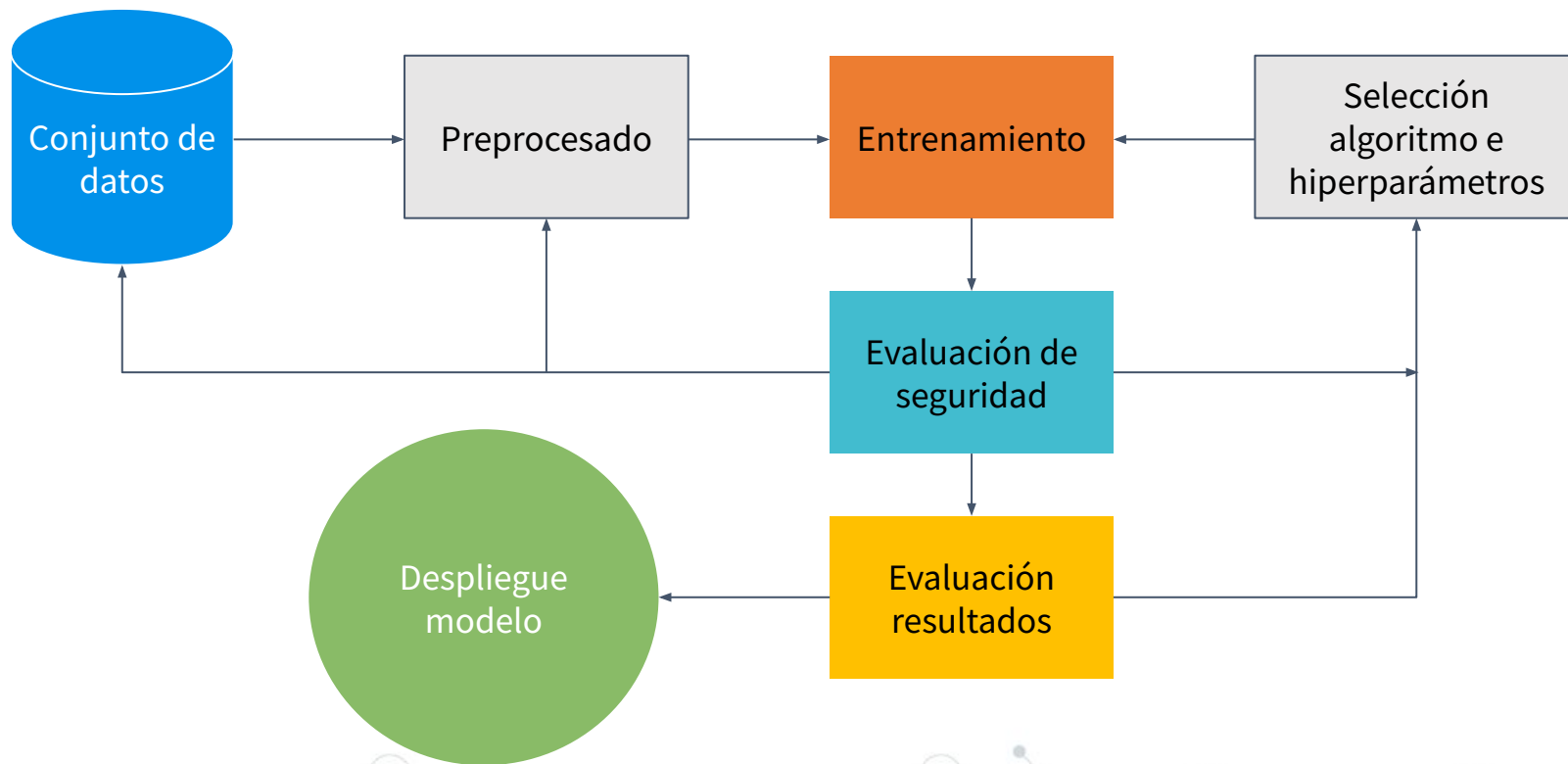
“

“If you use **machine learning**, there is the risk for exposure, even **though the threat does not currently exist in your space.**”

“The **gap** between **machine learning and security** is definitely there.”

“Through 2022, 30% of all AI cyberattacks will leverage **training-data poisoning**, AI **model theft**, or **adversarial samples** to attack AI-powered systems.”

Conclusiones



Siguientes pasos

 [@dawnsongtweets](https://twitter.com/dawnsongtweets)

 [@goodfellow_ian](https://twitter.com/goodfellow_ian)

 [@NicolasPapernot](https://twitter.com/NicolasPapernot)

 [@biggiobattista](https://twitter.com/biggiobattista)

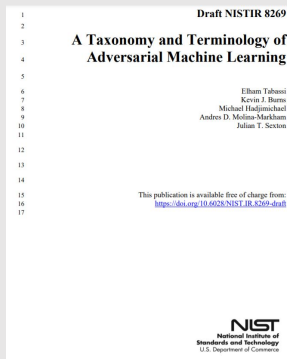
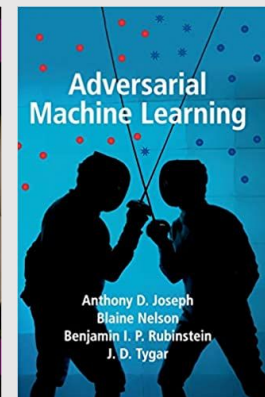
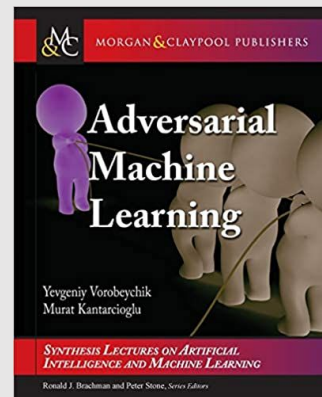
 [Nicholas Carlini](https://www.nicholas.carlini.me)

 [David Wagner](https://www.davidwagner.io)

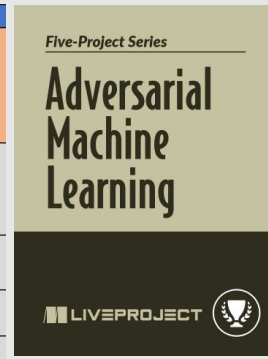
 [Anish Athalye](https://www.anishathalye.com)

 [Martín Abadi](https://www.martinabadi.com)

 [Aleksander Mądry](https://www.aleksander.majdry.com)



Reconnaissance	Initial Access
Acquire OSINT information: (Sub Techniques) 1. Anxiv 2. Public blogs 3. Press Releases 4. Conference Proceedings 5. GitHub Repository 6. Tweets	Pre-trained ML model with backdoor
ML Model Discovery (Sub Techniques) 1. Reveal ML model ontology – 2. Reveal ML model family –	Valid account
Gathering datasets	Phishing
Exploit physical environment	External remote services
Model Replication (Sub Techniques)	Exploit public facing



¡Gracias por su atención!

¿Alguna pregunta?



<https://github.com/jiep/adversarial-machine-learning>



[@MiguelHzBz](#)



[/in/josé-ignacio-escribano-pablos](#)



Después de proteger la infraestructura y los datos, deberías preocuparte por tus modelos de IA.

Introducción al Adversarial Machine Learning

